

Full Coding Activity Plan – CHATGPT Version

Phase 0 — Decide the smallest first experiment

Do **not** start with all datasets, all attacks, all defenses.

First run only:

Dataset: EMBER 2018 or your working EMBER subset

Model: LightGBM

Attack: Severi-style attack, one selector pair first

Sampling: random first

Defense:

1. No defense
2. Global Mahalanobis
3. Local GMM-Mahalanobis

Removal: top 1%, 3%, 5%

After that works, expand to PMSB sampling and other selector pairs.

Phase 1 — Prepare project structure

Create folders like this:

```
project/
|
|— data/
|   |— clean/
|   |— poisoned/
|   |— processed/
|
|— results/
|
|— shap/
|
|— gmm/
```

```
|  ├── defense/
|  └── plots/
|
|  ├── src/
|  │   ├── train_model.py
|  │   ├── attack_utils.py
|  │   ├── shap_utils.py
|  │   ├── gmm_maha_defense.py
|  │   ├── evaluation.py
|  │   └── plotting.py
|  └── notebooks/
|      └── 01_debug_gmm_defense.ipynb
```

For the first version, notebook is okay. Later, move stable functions into src/.

Phase 2 — Load your dataset

You need:

```
X_train_clean
y_train_clean
X_test
y_test
```

For poisoned experiments, you also need:

```
X_train_poisoned
y_train_poisoned
poison_mask_train
```

Where:

```
poison_mask_train[i] = True  # if sample i is poisoned
poison_mask_train[i] = False # otherwise
```

This mask is very important because you need to compute poison recall later.

Example structure:

```
import numpy as np
```

```
X_train_clean = np.load("data/processed/X_train_clean.npy")
```

```
y_train_clean = np.load("data/processed/y_train_clean.npy")
```

```
X_test = np.load("data/processed/X_test.npy")
```

```
y_test = np.load("data/processed/y_test.npy")
```

```
X_train_poisoned = np.load("data/poisoned/X_train_poisoned.npy")
```

```
y_train_poisoned = np.load("data/poisoned/y_train_poisoned.npy")
```

```
poison_mask_train = np.load("data/poisoned/poison_mask_train.npy")
```

Check shapes:

```
print(X_train_poisoned.shape)
```

```
print(y_train_poisoned.shape)
```

```
print(poison_mask_train.shape)
```

```
print("Poison count:", poison_mask_train.sum())
```

Expected:

```
X_train_poisoned.shape[0] == y_train_poisoned.shape[0]
```

```
X_train_poisoned.shape[0] == poison_mask_train.shape[0]
```

Phase 3 — Train baseline LightGBM on poisoned data

This is your attacked/backdoored classifier.

```
import lightgbm as lgb
```

```
from sklearn.metrics import accuracy_score, roc_auc_score
```

```
def train_lgbm(X_train, y_train):
```

```
    model = lgb.LGBMClassifier(
```

```
        n_estimators=500,
```

```
        learning_rate=0.05,
```

```
        num_leaves=64,
```

```
        subsample=0.8,
```

```
        colsample_bytree=0.8,
```

```
    random_state=42,  
    n_jobs=-1  
)  
model.fit(X_train, y_train)  
return model
```

```
model_poisoned = train_lgbm(X_train_poisoned, y_train_poisoned)
```

Evaluate clean accuracy:

```
y_pred = model_poisoned.predict(X_test)  
y_prob = model_poisoned.predict_proba(X_test)[:, 1]
```

```
clean_acc = accuracy_score(y_test, y_pred)  
clean_auc = roc_auc_score(y_test, y_prob)
```

```
print("Clean accuracy:", clean_acc)  
print("Clean ROC-AUC:", clean_auc)
```

Phase 4 — Evaluate ASR before defense

You need your watermarked malware test set:

```
X_test_watermarked_malware  
y_test_watermarked_malware
```

Usually:

```
true label = malware  
target wrong prediction = benign
```

If your label convention is:

```
0 = benign  
1 = malware
```

Then evasion / ASR means:

```
prediction == 0
```

Code:

```
def compute_asr(model, X_watermarked_malware):  
    pred = model.predict(X_watermarked_malware)  
    asr = np.mean(pred == 0)  
    return asr
```

```
asr_before = compute_asr(model_poisoned, X_test_watermarked_malware)  
print("ASR before defense:", asr_before)
```

This is your baseline attack success.

Phase 5 — Extract benign-labeled training samples

Your defense only inspects benign-labeled samples because clean-label poisoned samples are labeled benign.

```
benign_label = 0
```

```
benign_idx = np.where(y_train_poisoned == benign_label)[0]
```

```
X_benign_train = X_train_poisoned[benign_idx]  
poison_mask_benign = poison_mask_train[benign_idx]
```

```
print("Benign-labeled samples:", len(benign_idx))  
print("Poison inside benign-labeled:", poison_mask_benign.sum())
```

This gives you:

all benign-labeled samples
including clean benign + poisoned benign

Phase 6 — Compute SHAP vectors

Install/import SHAP:

```
import shap
```

Batch function:

```
def compute_shap_in_batches(model, X, batch_size=5000):
    explainer = shap.TreeExplainer(model)
    shap_batches = []

    for start in range(0, len(X), batch_size):
        end = start + batch_size
        X_batch = X[start:end]

        sv = explainer.shap_values(X_batch)

        # LightGBM binary classification may return list [class0, class1]
        if isinstance(sv, list):
            sv = sv[1] # malware/positive class

        shap_batches.append(sv)

        print(f"Computed SHAP {start}:{min(end, len(X))}")

    return np.vstack(shap_batches)
```

Run:

```
X_benign_shap = compute_shap_in_batches(
    model_poisoned,
    X_benign_train,
    batch_size=5000
)
```

```
print(X_benign_shap.shape)
```

Save it:

```
np.save("results/shap/X_benign_shap.npy", X_benign_shap)
np.save("results/shap/benign_idx.npy", benign_idx)
np.save("results/shap/poison_mask_benign.npy", poison_mask_benign)
```

For debugging, start with smaller subset:

```
debug_n = 20000
X_benign_train_debug = X_benign_train[:debug_n]
poison_mask_benign_debug = poison_mask_benign[:debug_n]
```

Phase 7 — PCA reduction

Use fixed 50 or 100 components first. Do not blindly use 95% variance if it keeps too many dimensions.

```
from sklearn.decomposition import PCA
from sklearn.preprocessing import StandardScaler

def reduce_with_pca(X, n_components=50):
    scaler = StandardScaler()
    X_scaled = scaler.fit_transform(X)

    pca = PCA(n_components=n_components, random_state=42)
    X_reduced = pca.fit_transform(X_scaled)

    print("Explained variance:", pca.explained_variance_ratio_.sum())

    return X_reduced, scaler, pca
```

Run:

```
X_shap_reduced, shap_scaler, shap_pca = reduce_with_pca(
    X_benign_shap,
    n_components=50
)
```

Save:

```
np.save("results/gmm/X_shap_reduced.npy", X_shap_reduced)
```

Later, test:

```
n_components = 25, 50, 100
```

But first use 50.

Phase 8 — Fit Global Mahalanobis baseline

Global Mahalanobis is basically GMM with $K = 1$.

```
from sklearn.mixture import GaussianMixture

def fit_gmm(X, n_components, covariance_type="diag", reg_covar=1e-6):
    gmm = GaussianMixture(
        n_components=n_components,
        covariance_type=covariance_type,
        reg_covar=reg_covar,
        random_state=42,
        n_init=3,
        max_iter=200
    )
    gmm.fit(X)
    return gmm
```

Fit global:

```
global_gmm = fit_gmm(
    X_shap_reduced,
    n_components=1,
    covariance_type="diag"
)
```

Phase 9 — Fit Local GMM with BIC

```
def fit_gmm_bic(X, k_range=range(1, 11), covariance_type="diag", reg_covar=1e-6):
    best_gmm = None
    best_bic = np.inf
    bic_scores = {}

    for k in k_range:
        print(f"Fitting GMM K={k}")
```



```

gmm = GaussianMixture(
    n_components=k,
    covariance_type=covariance_type,
    reg_covar=reg_covar,
    random_state=42,
    n_init=3,
    max_iter=200
)

gmm.fit(X)
bic = gmm.bic(X)

bic_scores[k] = bic
print(f"K={k}, BIC={bic}")

if bic < best_bic:
    best_bic = bic
    best_gmm = gmm

print("Best K:", best_gmm.n_components)
return best_gmm, bic_scores

```

Run:

```

local_gmm, bic_scores = fit_gmm_bic(
    X_shap_reduced,
    k_range=range(1, 11),
    covariance_type="diag",
    reg_covar=1e-6
)

```

Plot BIC:

```

import matplotlib.pyplot as plt

plt.figure(figsize=(6, 4))
plt.plot(list(bic_scores.keys()), list(bic_scores.values()), marker="o")
plt.xlabel("Number of GMM components K")
plt.ylabel("BIC")
plt.title("GMM-BIC model selection")
plt.grid(True)
plt.show()

```

Check cluster sizes:

```
cluster_labels = local_gmm.predict(X_shap_reduced)

unique, counts = np.unique(cluster_labels, return_counts=True)
print(dict(zip(unique, counts)))
```

If you see tiny clusters like 3 samples, 7 samples, etc., be careful. You may need fewer K, more regularization, or `covariance_type="diag"`.

Phase 10 — Compute Mahalanobis scores

```
def gmm_mahalanobis_scores(X, gmm):
    labels = gmm.predict(X)
    scores = np.zeros(len(X), dtype=float)

    for k in range(gmm.n_components):
        idx = np.where(labels == k)[0]

        if len(idx) == 0:
            continue

        Xk = X[idx]
        mu = gmm.means_[k]
        diff = Xk - mu

        if gmm.covariance_type == "diag":
            var = gmm.covariances_[k] + 1e-8
            dist = np.sum((diff ** 2) / var, axis=1)

        elif gmm.covariance_type == "full":
            cov = gmm.covariances_[k]
            cov_inv = np.linalg.pinv(cov)
            dist = np.einsum("ij,jk,ik->i", diff, cov_inv, diff)

        else:
            raise ValueError("Only diag and full covariance supported.")
```

```
scores[idx] = dist
```

```
return scores, labels
```

Global scores:

```
global_scores, global_labels = gmm_mahalanobis_scores(  
    X_shap_reduced,  
    global_gmm  
)
```

Local scores:

```
local_scores, local_labels = gmm_mahalanobis_scores(  
    X_shap_reduced,  
    local_gmm  
)
```

Phase 11 — Normalize scores

Option A — Global score normalization

For global Mahalanobis:

```
def zscore(scores):  
    return (scores - scores.mean()) / (scores.std() + 1e-8)
```

```
global_z = zscore(global_scores)
```

Option B — Cluster-wise z-score

For local Mahalanobis:

```
def clusterwise_zscore(scores, labels):  
    z = np.zeros_like(scores, dtype=float)  
  
    for k in np.unique(labels):  
        idx = np.where(labels == k)[0]
```

```
sk = scores[idx]

z[idx] = (sk - sk.mean()) / (sk.std() + 1e-8)
```

```
return z
```

Run:

```
local_z = clusterwise_zscore(local_scores, local_labels)
```

Optional robust version

```
def clusterwise_robust_zscore(scores, labels):
    z = np.zeros_like(scores, dtype=float)

    for k in np.unique(labels):
        idx = np.where(labels == k)[0]
        sk = scores[idx]

        median = np.median(sk)
        mad = np.median(np.abs(sk - median)) + 1e-8

        z[idx] = 0.6745 * (sk - median) / mad

    return z
```

Use later if normal z-score behaves weirdly.

Phase 12 — Diagnostic before removing anything

Before retraining, check if poison samples get higher scores.

```
def summarize_scores(name, scores, poison_mask):
    clean_scores = scores[~poison_mask]
    poison_scores = scores[poison_mask]

    print(f"\n{name}")
```

```
print("Clean mean:", clean_scores.mean())
print("Poison mean:", poison_scores.mean())
print("Clean p95:", np.percentile(clean_scores, 95))
print("Poison p95:", np.percentile(poison_scores, 95))
print("Clean p99:", np.percentile(clean_scores, 99))
print("Poison p99:", np.percentile(poison_scores, 99))
```

```
summarize_scores("Global Mahalanobis", global_z, poison_mask_benign)
summarize_scores("Local GMM-Mahalanobis", local_z, poison_mask_benign)
```

Plot distribution:

```
def plot_score_distribution(scores, poison_mask, title):
    plt.figure(figsize=(7, 4))
    plt.hist(scores[~poison_mask], bins=80, alpha=0.6, density=True, label="Clean
benign")
    plt.hist(scores[poison_mask], bins=80, alpha=0.6, density=True, label="Poisoned
benign")
    plt.xlabel("Suspicion score")
    plt.ylabel("Density")
    plt.title(title)
    plt.legend()
    plt.grid(True)
    plt.show()
```

```
plot_score_distribution(global_z, poison_mask_benign, "Global Mahalanobis score")
plot_score_distribution(local_z, poison_mask_benign, "Local GMM-Mahalanobis
score")
```

If the poison distribution is shifted right, good.

If not, the method may not detect that poison type.

That is not failure yet. It may be your failure analysis.

Phase 13 — Select suspicious samples

Top-p removal:

```
def get_top_percent_indices(scores, percent):
    n = len(scores)
    n_remove = int(np.ceil(n * percent / 100.0))
    return np.argsort(scores)[-n_remove:]
```

Evaluate removal quality without retraining:

```
def evaluate_removal(remove_local_idx, poison_mask_benign):
    removed_poison = poison_mask_benign[remove_local_idx].sum()
    total_poison = poison_mask_benign.sum()

    removed_clean = (~poison_mask_benign[remove_local_idx]).sum()
    total_clean = (~poison_mask_benign).sum()

    poison_recall = removed_poison / max(total_poison, 1)
    clean_fpr = removed_clean / max(total_clean, 1)

    return {
        "removed_total": len(remove_local_idx),
        "removed_poison": int(removed_poison),
        "removed_clean": int(removed_clean),
        "poison_recall": poison_recall,
        "clean_fpr": clean_fpr
    }
```

Run for several budgets:

```
for percent in [1, 3, 5]:
    remove_global = get_top_percent_indices(global_z, percent)
    remove_local = get_top_percent_indices(local_z, percent)

    print("\nRemoval budget:", percent)

    print("Global:")
    print(evaluate_removal(remove_global, poison_mask_benign))

    print("Local:")
    print(evaluate_removal(remove_local, poison_mask_benign))
```

This is your first important result.

Phase 14 — Map removal indices back to original training set

`remove_local_idx` indexes `X_benign_train`, not the full training set.

Map back:

```
remove_global_original_idx = benign_idx[remove_global]
remove_local_original_idx = benign_idx[remove_local]
```

Create sanitized dataset:

```
def sanitize_training_set(X_train, y_train, remove_original_idx):
    keep_mask = np.ones(len(X_train), dtype=bool)
    keep_mask[remove_original_idx] = False

    X_sanitized = X_train[keep_mask]
    y_sanitized = y_train[keep_mask]

    return X_sanitized, y_sanitized, keep_mask
```

Example:

```
X_train_local_san, y_train_local_san, keep_mask_local = sanitize_training_set(
    X_train_poisoned,
    y_train_poisoned,
    remove_local_original_idx
)
```

Phase 15 — Retrain after defense

```
model_local_defended = train_lgbm(
    X_train_local_san,
    y_train_local_san
)
```

Evaluate clean performance:

```
def evaluate_clean(model, X_test, y_test):
    pred = model.predict(X_test)
    prob = model.predict_proba(X_test)[: , 1]

    return {
        "accuracy": accuracy_score(y_test, pred),
        "roc_auc": roc_auc_score(y_test, prob)
    }
```

```
clean_metrics_local = evaluate_clean(model_local_defended, X_test, y_test)
print(clean_metrics_local)
```

Evaluate ASR after defense:

```
asr_after_local = compute_asr(
    model_local_defended,
    X_test_watermarked_malware
)
```

```
print("ASR before:", asr_before)
print("ASR after local defense:", asr_after_local)
print("Defense Survival Rate:", asr_after_local / max(asr_before, 1e-8))
```

Repeat for global:

```
X_train_global_san, y_train_global_san, keep_mask_global = sanitize_training_set(
    X_train_poisoned,
    y_train_poisoned,
    remove_global_original_idx
)
```

```
model_global_defended = train_lgbm(
    X_train_global_san,
    y_train_global_san
)
```

```
asr_after_global = compute_asr(
    model_global_defended,
    X_test_watermarked_malware
)
```

```
clean_metrics_global = evaluate_clean(model_global_defended, X_test, y_test)
```



```
print("Global clean:", clean_metrics_global)
print("Global ASR after:", asr_after_global)
print("Global DSR:", asr_after_global / max(asr_before, 1e-8))
```

Phase 16 — Create one result table

Use pandas:

```
import pandas as pd
```

```
results = []
```

```
def add_result(defense_name, removal_percent, removal_eval, clean_metrics,
asr_before, asr_after):
    results.append({
        "defense": defense_name,
        "removal_percent": removal_percent,
        "removed_total": removal_eval["removed_total"],
        "removed_poison": removal_eval["removed_poison"],
        "removed_clean": removal_eval["removed_clean"],
        "poison_recall": removal_eval["poison_recall"],
        "clean_fpr": removal_eval["clean_fpr"],
        "clean_accuracy": clean_metrics["accuracy"],
        "clean_roc_auc": clean_metrics["roc_auc"],
        "asr_before": asr_before,
        "asr_after": asr_after,
        "defense_survival_rate": asr_after / max(asr_before, 1e-8)
    })
```

For each defense/removal budget, append result.

At the end:

```
df_results = pd.DataFrame(results)
df_results.to_csv("results/defense/gmm_maha_results.csv", index=False)
df_results
```

Phase 17 — Add Isolation Forest baseline

Fit on the same SHAP/PCA representation:

```
from sklearn.ensemble import IsolationForest
```

```
iso = IsolationForest(  
    n_estimators=200,  
    contamination="auto",  
    random_state=42,  
    n_jobs=-1  
)
```

```
iso.fit(X_shap_reduced)
```

```
# Lower decision_function = more anomalous
```

```
iso_raw = iso.decision_function(X_shap_reduced)
```

```
# Convert so higher = more suspicious
```

```
iso_score = -iso_raw
```

```
iso_z = zscore(iso_score)
```

Then use the same removal/evaluation functions:

```
remove_iso = get_top_percent_indices(iso_z, percent=1)
```

```
eval_iso = evaluate_removal(remove_iso, poison_mask_benign)
```

```
remove_iso_original_idx = benign_idx[remove_iso]
```

Retrain and evaluate the same way.

This gives fair comparison:

same representation

same removal budget

different scoring method

Phase 18 — Add raw-feature ablation

Later, test whether SHAP space helps.

Instead of:

X_benign_shap

Use:

X_benign_train

Pipeline:

```
X_raw_reduced, raw_scaler, raw_pca = reduce_with_pca(  
    X_benign_train,  
    n_components=50  
)
```

```
local_gmm_raw, bic_raw = fit_gmm_bic(X_raw_reduced)  
raw_scores, raw_labels = gmm_mahalanobis_scores(X_raw_reduced, local_gmm_raw)  
raw_z = clusterwise_zscore(raw_scores, raw_labels)
```

Then evaluate.

Now you can compare:

Raw Local Mahalanobis

vs

SHAP Local Mahalanobis

This tells you whether explanation space is useful.

Phase 19 — Add sampling strategy loop

Once the single experiment works, wrap everything in a loop.

Configuration:

```
experiment_configs = [  
    {  
        "attack": "combined",  
        "sampling": "random",  
        "poison_rate": 0.01,  
        "path": "data/poisoned/combined_random_1pct/"
```

```

    },
    {
        "attack": "combined",
        "sampling": "pmsb_contribution",
        "poison_rate": 0.01,
        "path": "data/poisoned/combined_pmsb_contribution_1pct/"
    },
    {
        "attack": "minpopulation",
        "sampling": "random",
        "poison_rate": 0.01,
        "path": "data/poisoned/minpop_random_1pct/"
    },
]

```

For each config:

```
for cfg in experiment_configs:
```

```
    print("Running:", cfg)
```

```

        # 1. load poisoned data
        # 2. train poisoned model
        # 3. compute ASR before
        # 4. extract benign samples
        # 5. compute/load SHAP
        # 6. PCA
        # 7. fit global/local/IF
        # 8. remove top p%
        # 9. retrain
        # 10. save result row

```

Important: cache SHAP values. Do not recompute if file exists.

```
import os
```

```
shap_path = f"results/shap/{cfg['attack']}_{cfg['sampling']}_{cfg['poison_rate']}.npy"
```

```
if os.path.exists(shap_path):
```

```
    X_benign_shap = np.load(shap_path)
```

```
else:
```

```
    X_benign_shap = compute_shap_in_batches(model_poisoned, X_benign_train)
```

```
    np.save(shap_path, X_benign_shap)
```

Phase 20 — Plots you should generate

Plot 1 — score distribution

For each defense:

clean benign score distribution

poisoned benign score distribution

Plot 2 — ASR before/after defense

Bar chart:

x-axis: sampling strategy

y-axis: ASR

bars: before defense, after global, after local, after IF

Plot 3 — poison recall vs clean FPR

Scatter plot:

x-axis: clean FPR

y-axis: poison recall

point color: defense

point shape: sampling strategy

This one is very useful.

Plot 4 — Defense Survival Rate

lower = better defense

higher = attack survives

Plot 5 — selected K from BIC

Show whether PMSB sampling changes natural cluster structure.

Phase 21 — Final result table columns

Your final CSV should have columns like:

dataset
model
attack_strategy
sampling_strategy
poison_rate
defense
representation
pca_components
gmm_k
removal_percent
poison_recall
clean_fpr
clean_accuracy_before
clean_accuracy_after
clean_auc_before
clean_auc_after
asr_before
asr_after
defense_survival_rate
runtime_seconds

This table will save your life later.

Phase 22 — Minimum paper-worthy experiment

To avoid death by comparisons, first produce this:

Attack strategies:

Combined

MinPopulation

Sampling:

Random
PMSB-contribution

Defense:
No defense
Isolation Forest
Global Mahalanobis
Local GMM-Mahalanobis

Removal:
top 1%
top 3%

Dataset:
EMBER only

Model:
LightGBM only

That is already:

$2 \times 2 \times 4 \times 2 = 32$ defended runs

Actually enough.

Do not start with 300 runs. That is how research becomes a haunted house.

Phase 23 — How to interpret outcomes

Case A — Local GMM beats Global

You can say:

Local benign subpopulation modeling improves poison removal compared with one global benign distribution.

Nice.

Case B — Local does not beat Global

Still useful:

Poisoned samples are not simply local covariance anomalies; their detectability depends more on selector/sampling behavior.

Still a result.

Case C — PMSB has high ASR but also high poison recall

Interpretation:

PMSB improves attack strength but leaves a stronger defense footprint.

Very good.

Case D — PMSB has high ASR and low poison recall

Interpretation:

PMSB improves both effectiveness and stealth, making it a stronger defense-aware attack.

Also very good.

Case E — Combined survives everything

Interpretation:

Combined-style feature-value selection remains difficult because it embeds watermark values in goodwill-supported regions.

Also very good.

So you have many ways to win.

Your actual first coding target

For now, make one notebook that does only this:

01_debug_gmm_defense.ipynb

Input:

- one poisoned dataset

Output:

- table comparing:

 - Global Mahalanobis

 - Local GMM-Mahalanobis

 - Isolation Forest

For:

- top 1%, 3%, 5% removal

Metrics:

- poison recall

- clean FPR

- ASR after defense

- clean accuracy after defense

When that works, you can expand.

That is your first checkpoint. Tiny robot first. Army of robots later.

Full Coding Activity Plan – Perplexity Version

Project phases

Phase 0

Set up a clean, reproducible experiment structure before changing any attack logic. This matters because later you will compare many attack strategies, sampling strategies, and defenses, and inconsistent file naming will ruin the study faster than coding bugs.
`shap.readthedocs+1`

Create directories like:

- data/
- artifacts/models/
- artifacts/shap/
- artifacts/poisoned_sets/
- artifacts/defense_scores/
- results/tables/
- results/plots/
- src/

Inside src/, separate files by purpose:

- train_model.py
- attack_generation.py
- sampling_strategies.py
- shap_utils.py
- gmm_defense.py
- evaluate.py
- run_experiment.py

This is better than keeping everything only in notebooks because the final factorial study will require repeated batch runs.

Phase 1

Reproduce your existing clean baseline first. Train the original malware classifier on the clean dataset and save:

- model file
- train/validation/test indices
- clean test metrics
- feature names

Do not start defense coding before this clean baseline is stable, because every later ASR and defense result depends on it. [shap.readthedocs](#)

Code activity:

1. Load EMBER/BODMAS split.
2. Train your baseline model.
3. Save predictions on clean test set.
4. Save a metadata JSON with dataset name, seed, model hyperparameters, and split hashes.

Output files:

- artifacts/models/clean_lgbm_seed42.pkl
- results/tables/clean_baseline_metrics.csv

Attack reproduction

Phase 2

Port your current poisoning pipeline into callable functions. Your repo already contains multiple attack notebooks and `sampling_utils.py`, so the first job is to turn those into consistent functions instead of notebook-only logic.

You already have these sampling functions implemented:

- feature-based distance
- distribution-based distance
- SHAP contribution distance
- Mahalanobis distance
- cosine similarity

- Jaccard distance
- Wasserstein distance

Code activity:

1. Extract the attack generation logic from notebooks into:
 - a. `generate_watermark(...)`
 - b. `select_poison_candidates(...)`
 - c. `inject_watermark(...)`
 - d. `build_poisoned_dataset(...)`
2. Keep the same seeds and poison counts you used before.
3. Save poisoned train sets to disk with a manifest.

Suggested poisoned dataset naming:

- `ember_combined_random_pr0.01_seed42.parquet`
- `ember_combined_mahalanobis_pr0.01_seed42.parquet`

Each poisoned dataset should also save:

- poisoned sample indices
- original benign indices selected
- watermark features
- watermark values
- attack type
- sampling strategy
- poison rate

This metadata is essential later for poison recall evaluation.

SHAP extraction

Phase 3

Once attacks are reproducible, compute SHAP representations for the benign-labeled training samples. For tree models, SHAP TreeExplainer is the natural choice and is efficient for LightGBM/XGBoost-style models.[shap.readthedocs+1](#)

Code activity:

1. Load a poisoned training dataset.
2. Retrain the attacked model on that poisoned set.

3. Identify all benign-labeled training samples.
4. Compute SHAP values for those benign-labeled samples only.
5. Save:
 - a. SHAP matrix
 - b. sample IDs
 - c. labels
 - d. poison indicator
 - e. feature names

Important implementation note: for binary classification, SHAP outputs can differ in shape depending on model/wrapper, so normalize output shape early and save a single consistent 2D matrix.[github+1](#)

Output files:

- artifacts/shap/ember_combined_mahalanobis_pr0.01_seed42_benign_shap.npz
- artifacts/shap/..._metadata.csv

GMM-BIC defense prototype

Phase 4

Now build the defense itself. Start with a minimal version before fancy ablations.

The minimum practical pipeline is:

1. input benign SHAP matrix
2. optional standardization
3. optional PCA
4. fit several GMMs
5. choose best by BIC
6. assign each point to a component
7. compute anomaly score per point
8. remove suspicious points
9. retrain and evaluate `scikit-learn`

Code activity in `gmm_defense.py`:

- `preprocess_for_gmm(X, use_scaler=True, use_pca=True, pca_var=0.95)`
- `select_best_gmm(X, k_range, covariance_types)`
- `assign_components(gmm, X)`

- `compute_component_mahalanobis(gmm, X, labels)`
- `flag_suspicious_points(scores, mode='zscore' or 'top_p')`
- `sanitize_dataset(...)`

Practical implementation choices

Start simple:

- `covariance_type` in ['diag', 'tied', 'full']
- `n_components` = 1..10
- `reg_covar` = 1e-6 or 1e-5 to stabilize covariance estimation [github](#)+1
- PCA first if dimensionality is high

This is especially important because high-dimensional SHAP vectors can make full covariance unstable; scikit-learn explicitly exposes `reg_covar` and covariance-type choices for this reason. [scikit-learn](#)+1

Scoring choice

You have two good initial scoring options:

- **Option A:** model log-likelihood / negative log-likelihood from GMM
- **Option B:** Mahalanobis distance to assigned component center

Since your story is specifically about local subpopulation anomaly scoring, start with **Mahalanobis within component**. The Mahalanobis distance is exactly the right local-distance concept when covariance matters. [arxiv](#)+1

Implementation note: be careful whether a library method returns distance or squared distance. Some related sklearn covariance utilities historically returned squared distance, so be explicit and test your own formula. [github](#)

First validation experiments

Phase 5

Do not immediately run the full paper experiment. First do one controlled sanity test:

- one attack type, e.g. Combined
- one sampling strategy, e.g. random
- one poison rate, e.g. 1%

- one seed

Code activity:

1. Train attacked model.
2. Compute SHAP.
3. Run GMM-BIC.
4. Remove suspicious points.
5. Retrain model.
6. Compare:
 - a. clean accuracy before/after
 - b. ASR before/after
 - c. poison recall
 - d. clean benign false positive rate

If this pipeline works end-to-end once, only then scale up.

Save outputs:

- results/tables/sanity_check_gmm.csv
- artifacts/defense_scores/...csv

Defense threshold tuning

Phase 6

After the sanity run, tune how you decide what to remove.

You should not assume one threshold works universally. Evaluate at least:

- top 1%, 3%, 5% removal
- z-score threshold 2.0, 2.5, 3.0, 3.5 within component
- optional likelihood percentile thresholdsarxiv+1

Code activity:

1. For one or two benchmark attack settings, sweep thresholds.
2. Plot:
 - a. poison recall vs clean false positive rate
 - b. ASR after defense vs removed benign fraction
3. Choose one primary thresholding method for the main paper.

This stage matters because a defense can look strong simply by deleting too many benign points.

Baseline defenses

Phase 7

Implement comparison baselines before the final factorial study. You already discussed:

- Isolation Forest
- Spectral Signatures
- Nested Training or simplified disagreement
- your GMM-Mahalanobis

Code activity:

- `baseline_isoforest.py`
- `baseline_spectral.py`
- `baseline_disagreement.py`

Each baseline should expose the same interface:

- `fit_score(X, metadata)`
- returns sample-level suspiciousness scores

This lets `run_experiment.py` call any defense the same way.

Full experiment matrix

Phase 8

Now run the complete study.

Suggested factors:

- attack strategy: Combined, MinPopulation, CountAbsSHAP
- sampling strategy: random, feature-based, distribution-based, SHAP-distance, Mahalanobis, cosine, Jaccard, Wasserstein
- poison rate: 1%, 2%, 5%
- seeds: at least 3

- defenses: none, Isolation Forest, Spectral, disagreement, GMM-Mahalanobis

Code activity:

1. Create a config list of all combinations.
2. Loop over each configuration.
3. For each:
 - a. generate or load poisoned set
 - b. train attacked model
 - c. compute attacked metrics
 - d. run each defense
 - e. retrain post-defense
 - f. save all outputs row-wise

Master results schema:

- dataset
- model
- seed
- attack_type
- sampling_type
- poison_rate
- defense
- clean_acc_before
- clean_acc_after
- asr_before
- asr_after
- poison_recall
- benign_fpr
- num_removed
- runtime_sec

Save to one long CSV:

- results/tables/main_results_long.csv

Analysis and ablations

Phase 9

Once main results exist, run ablations to explain the defense, not just report numbers.

Recommended ablations:

- raw features vs SHAP space
- no PCA vs PCA
- GMM covariance type: diag vs full
- likelihood score vs Mahalanobis score
- BIC-selected K vs fixed K
- with and without scalingscikit-learn+1

Code activity:

1. Fix one attack family and a few representative samplers.
2. Run these ablations.
3. Save smaller ablation tables and line plots.

This is where you show which part of your method actually matters.

Visual diagnostics

Phase 10

You need diagnostic plots for interpretation and the paper.

Create:

- BIC vs number of components plot
- histogram of Mahalanobis scores for clean vs poisoned benign samples
- t-SNE/UMAP of SHAP space colored by GMM component and poison label
- tradeoff curves for threshold tuning
- heatmap: sampling strategy × defense effectiveness

These plots are not decoration; they explain why certain samplers are easier or harder to detect.

Paper-ready checks

Phase 11

Before writing, verify all claims are reproducible.

Checklist:

1. Re-run selected key experiments from scratch with fresh outputs.
2. Confirm seeds are fixed.
3. Confirm poison indices saved match recall calculation.
4. Confirm ASR definition is consistent everywhere.
5. Confirm no leakage between train/test when computing SHAP or PCA.

This stage is boring but crucial. A single leakage bug can invalidate the paper.

What the actual coding order should be

Here is the most practical order to code:

1. Clean baseline training script
2. Attack-generation refactor from notebooks
3. Unified dataset manifest saving
4. SHAP extraction utility
5. Minimal GMM-BIC selection function
6. Mahalanobis scoring function
7. Simple top-p removal and retraining loop
8. One sanity experiment end-to-end
9. Threshold sweep
10. Baseline defenses
11. Full factorial runner
12. Plotting and ablations

That order minimizes debugging pain because each step depends only on the previous one.

Minimal pseudocode structure

python

```
# 1. train clean model
model = train_model(X_train_clean, y_train_clean)

# 2. build poisoned dataset
poisoned = build_poisoned_dataset(
    X_train, y_train,
    attack_type="combined",
    sampling_type="mahalanobis",
    poison_rate=0.01
)
```

```

# 3. train attacked model
atk_model = train_model(poisoned.X, poisoned.y)

# 4. compute benign SHAP space
X_benign = poisoned.X[poisoned.y == 0]
shap_benign = compute_shap(atk_model, X_benign)

# 5. optional preprocessing
Z = preprocess_for_gmm(shap_benign, scale=True, pca=True)

# 6. fit GMMs and choose by BIC
best_gmm = select_best_gmm(Z, k_range=range(1, 11),
                           covariance_types=["diag", "full"])

# 7. assign cluster/component
comp = best_gmm.predict(Z)

# 8. score each benign point locally
scores = mahalanobis_per_component(Z, best_gmm, comp)

# 9. flag suspicious benign samples
flag_mask = threshold_scores(scores, mode="zscore", tau=3.0)

# 10. remove and retrain
sanitized = remove_flagged_benign_samples(poisoned, flag_mask)
def_model = train_model(sanitized.X, sanitized.y)

# 11. evaluate
metrics = evaluate_attack_and_defense(def_model, test_clean, test_backdoor)

```

The GMM model selection part is standard sklearn usage, and BIC is computed directly from fitted models on the training matrix.[github+2](#)

Time estimate

A realistic estimate for someone already familiar with the repo:

- refactor notebooks to scripts: 2–4 days
- first working GMM defense: 1–2 days
- threshold tuning and bug fixing: 2–3 days

- full experiment runner and logging: 2–4 days
- plots and ablations: 2–3 days

So the full coding activity from start to paper-ready experiments is roughly **2–3 weeks of focused work**, not because GMM is hard, but because experiment engineering takes time.scikit-learn+1

My strongest advice

Do **not** begin by coding every sampler, every defense, and every dataset at once. Start with:

- EMBER
- LightGBM
- Combined attack
- random sampler
- one seed
- your GMM-Mahalanobis defense

Make that path perfect first. Then scale.

Would you like me to turn this into a **task checklist with filenames/functions**, matched directly to your current GitHub repo structure?